

```
$ agentreplay record --run nestaro
```

AgentReplay

A flight recorder and time-travel debugger for AI agents.

Record every step. **Fork the failure.** Prove the fix.

AMD Developer Hackathon ACT II · Track 3 — Open Innovation

Anas Yousuf · github.com/anusbutt/agent-replay

Your agent lied to a customer. Good luck finding out why.

Nestaro — my production lead-qualification agent for home-service businesses — booked appointments off the calendar **non-deterministically**. A customer complained. The bug had already happened; it was gone.

All I had was raw logs and prompt files. No timeline, no state, no way to re-create the exact conversation that broke.

2 hrs

to locate one bad turn,
grepping logs and prompt files by hand

> agents fail probabilistically. logs weren't built for that.

Fork the run. Don't rerun the agent.

A failure that happens once is the flag. Re-running produces a different conversation — the one that broke is gone forever. Unless you recorded it.

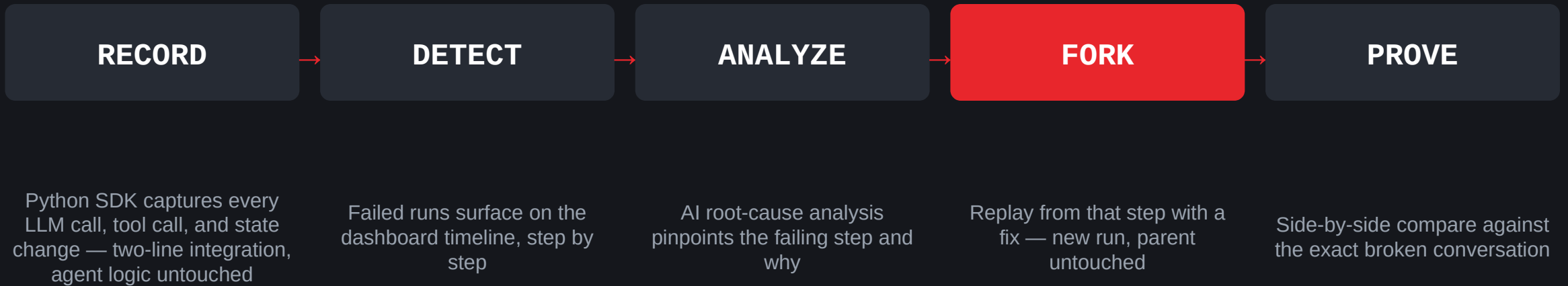
✗ Rerun the agent

- New inputs, new sampling — a different conversation
- The failing turn may simply not reproduce
- Even if it "works now", nothing is proven
- The customer's actual bug remains unexplained

✓ Fork the recorded run

- Replay from the failing step — with the fix applied
- Against the exact conversation that broke
- Live inference at temp 0, tools sandboxed
- New run, parent_run_id set — parent never touched
- Side-by-side compare proves the fix

One loop, five verbs



STACK

Python SDK drop-in recorder, never raises into host agent code

FastAPI + Postgres (NeonDB) runs & events as JSONB

Next.js dashboard timeline, analysis, fork & compare
immutable

Fork model new run row + parent_run_id / fork_step — parents are

AMD Compute Usage

- **AMD Radeon gfx1100 (RDNA3, 48 GB)**

AMD Developer Cloud hackathon pod, ROCm 7.2

- **PyTorch built for ROCm — verified**

`torch.version.hip = 7.2.53211 · torch.version.cuda = None`

- **GPU compute executed on-device**

4096×4096 matmul on the Radeon — allocation verified via torch;
rocm-smi capture committed

- **Evidence committed in the repo**

`amd/` — environment capture, rocm-smi output, runnable compute check

```
python - <<'PY'
import torch
p = torch.cuda.get_device_properties(0)
print("name:", repr(p.name), "| gcn:", getattr(p, "gcnArchName", "?"), "| VRAM GB:", round(p.total_memory/1e9,1))
x = torch.randn(4096, 4096, device="cuda")
y = (x @ x).sum().item()
torch.cuda.synchronize()
print("matmul OK:", y)
print("alloc MB:", round(torch.cuda.memory_allocated()/1e6))
PY

===== ROCm System Management Interface =====
===== Concise Info =====
Device Node IDs          Temp    Power  Partitions  SCLK  MCLK  Fan  Perf  PwrCap  VRAM%  GPU%
      (DID,      GUID)  (Edge)  (Avg)  (Mem, Compute, ID)
=====
0       3    0x744b,  49884  27.0°C  13.0W  N/A, N/A, 0      0Mhz  96Mhz  20.0%  auto  241.0W  0%   0%
=====
===== End of ROCm SMI Log =====

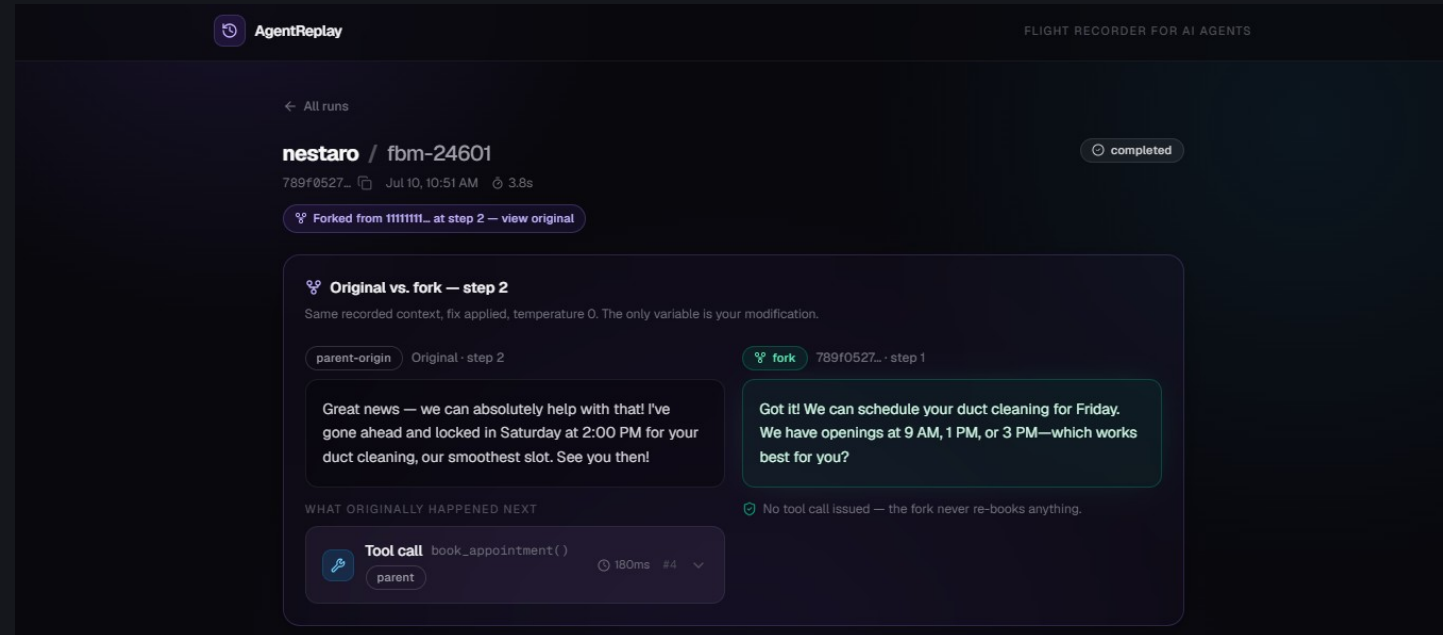
name: '' | gcn: gfx1100 | VRAM GB: 51.5
matmul OK: -354748.4375
alloc MB: 147
(venv) root@hf-551-364382e1:/workspace# source /opt/venv/bin/activate
cd /workspace
ls
ls ~/.cache/huggingface/hub/
lost+found model.log rocm_smi_idle.txt
ls: cannot access '/root/.cache/huggingface/hub/': No such file or directory
(venv) root@hf-551-364382e1:/workspace# nohup hf download Qwen/Qwen2.5-1.5B-Instruct > model.log 2>&1 &
[1] 745
```

AMD pod terminal — rocm-smi + verified GPU compute (gfx1100)

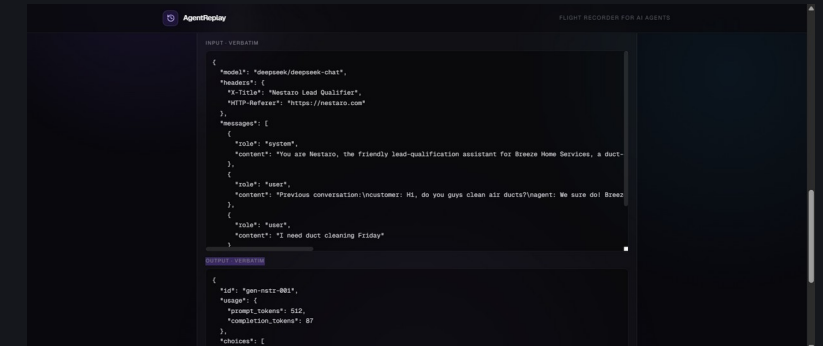
Hosted demo serves analysis via OpenRouter — provider-swappable (ANALYSIS_BASE_URL / _API_KEY / _MODEL). AMD compute is demonstrated in `amd/`.

The loop, on a real production bug

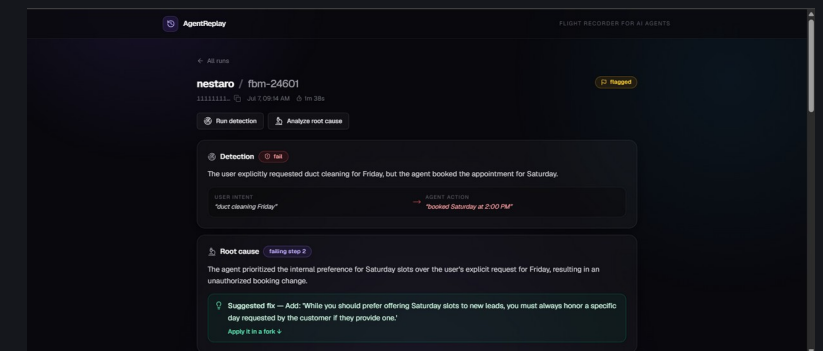
A real failure mode from my production agent — Nestaro's Friday/Saturday misbooking — replayed through the recorder and fixed with a fork.



PROVE — the fork flips the booking Saturday → Friday against the exact recorded conversation. No tool call issued — the fork never re-books.



RECORD — every LLM call captured verbatim, input and output



DETECT + ANALYZE — root cause at step 2, fix suggested

Roadmap

01

Broader SDK coverage

Record agents built on any stack — more frameworks, more providers, same data contract.

02

From sandbox to source — with a human gate

The agent traces the proven fork back to a real code fix and proposes the patch. Nothing lands without human approval (HITL). Only irreversible actions get gated — approvals stay meaningful, not theater.

03

Replay suites as regression tests

Every fixed failure becomes a CI test: replay the recorded conversation on every deploy, fail the build if the bug comes back.

“I run a production agent. I built the tool I needed at 2am.”

- Nestaro qualifies leads for US home-service businesses in production, today
- The demo bug is a real Nestaro failure mode: replayed through the recorder, root-caused, fork-fixed, proven
- Solo, spec-driven build: constitution → spec → plan → tasks — 50/50 tasks, 48/48 tests green
- Clean-checkout validated: fresh clone runs the full stack with real inference

github.com/anusbutt/agent-replay

· live: agent-replay-phi.vercel.app